

Repository Pattern

SWEN1 | BIF3

What is the Repository Pattern?

- Design pattern for abstracting database access
- Separates business logic from persistence logic
- Used to encapsulate CRUD operations and hide data access technology

Why use it?

✅ Advantages:

- Decouples application logic from database
- Improves testability (mock repositories)
- Centralizes data access logic
- Supports clean architecture (e.g. Single-Responsibility Pattern (SRP), Dependency-Inversion Pattern (DIP))

When **not** needed:

- Simple applications with direct CRUD
- No complex logic in data access
- One-to-one mapping between tables and objects

Repository in Java (Example)

```java
public interface UserRepository {
  Optional<User> findById(int id);
  List<User> findAll();
  void save(User user);
  void delete(int id);
}
```

```java
public class JdbcUserRepository implements UserRepository {
  private final DataSource ds;

  public JdbcUserRepository(DataSource ds) {
    this.ds = ds;
  }

  public void save(User user) {
    // JDBC insert logic here
  }
}
```

Repository in C# (Example)

```
public interface IUserRepository {
  User? GetById(int id);
  IEnumerable<User> GetAll();
  void Add(User user);
  void Remove(int id);
}
```

```
public class AdoNetUserRepository : IUserRepository {
  private readonly string _connectionString;

  public AdoNetUserRepository(string connectionString) {
    _connectionString = connectionString;
  }

  public void Add(User user) {
    // ADO.NET logic
  }
}
```

When to use it in your project

- If your app contains multiple layers (e.g. UI <--> Service <--> Persistence)
- If you plan to write unit tests for logic without accessing the database
- If the data layer is expected to grow or change (e.g. switch from SQL to NoSQL

Further Reading

- Martin Fowler on Repository Pattern
- Microsoft Docs: Repository Pattern in .NET

Where does it fit?

This pattern **extends the topics** from:

- **Connect to Database**: Encapsulate JDBC/ADO.NET calls
- **CRUD Operations**: Provide abstraction around basic operations
- Can also support **SOLID principles** if used well